SKALA

KV cache 최적화 기술 평가

Course	AI
Number	210
Duration	1.5 DAYS
Status	In Progress
Type	Activity
Last Updated	August 12, 2026
Copyright	© 2025-2026. Keejoo Bae for SK AX / SKALA. All rights reserved.

2 more properties

- 실습 목표
- 주제
- KV cache Optimization
  - Doc Pool
  - GUIDE
    - A. Agent 정의(안)
    - B. 설계
    - C. 평가 관점 및 기준
    - D. 그래프 설계(안)
    - E. 평가 보고서
  - Deliverables
  - Final Reminders

실습 목표

- LangGraph 기반 Multi Agent + Agentic RAG 설계 및 개발
- 외부 정보 검색 및 요약하는 도구 정의
- 하나의 기술을 도메인, 시장, 이해관계자 등 복수 관점에서 중립적인 관점에서 비교 평가

주제

> **KV cache 최적화 기술 다관점 평가 - SW 압축 vs HW 메모리 접근**
>
> - KV cache라는 하나의 병목을 상반된 접근으로 푸는 두 진영의 기술을 각각 조사하고
> - 도메인, 시장성, 이해관계자 반응 등의 관점에서 어떻게 평가받고 있는지 비교 분석하여
> - 다관점 평가 보고서를 작성하는 Agentic RAG 설계 및 구현

▾ KV cache Optimization

왜 KV cache가 문제인가

- LLM은 문장을 한 토큰씩 생성하면서, 앞서 계산한 Key-Value를 저장해 두고 재사용함 ⇒ KV cache
  - 다시 계산하는 낭비를 없애주는 고마운 장치이지만
  - 문맥이 수천~수백만 토큰으로 길어지면 저장해야 할 어텐션은 선형으로 늘어나
  - 가속기의 HBM 용량을 빠르게 소진하게 됨
- KV cache는 "연산 병목"을 "메모리 병목"으로 바꿔 놓은 셈
- 이 문제를 푸는 접근은 현재...
  - SW 관점 : 데이터를 작게 만들자
  - HW 관점 : 담을 공간을 넓히자

  ← 대립적으로는 보이나, 실제로는 함께 쓰이는 경우 많음 (e.g., 데이터를 작게 만들면 넓은 공간으로 옮기기 쉬워짐)

진영1 - SW : 데이터를 작게 만들자

KV cahce 자체 크기를 줄이는 것

- 기존 HW(GPU/HBM) 위에서 SW만으로도 즉시 적용
- 다만 압축률을 높이면 정확도가 떨어질 위험 부담
- 아키텍처 변형(MLA)은 강력하지만 이를 위한 최적화된 서빙 위한 시스템 복잡도 고려

1. 압축 양자화 - 이미 만들어진 cache를 사후에 줄이기
   - Google, TurboQuant : 숫자를 더 적은 비트로 저장하는 방식, 어텐션 처리 속도를 최대 8배까지 끌어오렸다고 보고
   - NVIDIA, KVTC : 이미지/영상 압축과 유사한 변환 부호화 기법, 최대 20배 압축하면서 추론 정확도 유지한다고 보고
2. 아키텍처 개선 - 애초에 작게 나오도록 구조 바꾸기
   - 사후 압축이 아니라 어텐션 구조 자체를 재설계
   - DeepSeek, MLA (Multi-head Latent Attention) : KV를 저차원 잠재 공간으로 압축, KV cache 93.3% 줄이면서 표준 어텐션 보다 나은 성능으로 보고

진영2 - HW : 담을 공간을 넓히자

원본은 그대로 두면서 "어디에 담느냐"를 바꿈

- KV cache를 비싼 HBM 밖으로 옮겨, 더 넓고 값싼 메모리/스토리지 계층에 두는 것
- 정확도 손실 없이 대용량 확장 가능하나, 새로운 인프라 필요 및 잔존하는 전송 지연 이슈

- 딜레마 :
  - cache를 GPu에 유지하면 다른 사용자 몫의 메모리를 잡아먹고
  - 버리면 재계산 비용이 크고, 외부로 옮기면 전송 오버헤드 생김

  → '옮기기'를 빠르게 만드는데 집중
- NVIDIA, CMX :
  - KV cache를 NVMe 스토리지로 확장하는 플랫폼
  - 4계층 메모리 위계를 두어 NVMe에 상주하는 KV cache를 컨텍스트 메모리 주소 공간의 일부로 만듦
- CXL 메모리 방식 :
  - 일부 업체는 KV cache를 NVMe 플래시로 보내는 대신
  - CXL 메모리로 처리하는 제품 개발 중

▸ Doc Pool

GUIDE

A. Agent 정의(안)

- 아래 표를 참고하여 필요 에이전트 정의, 설계 목적에 따라 추가/변경 가능

에이전트	역할	RAG 여부	내용
🔍 기술 조사 에이전트	기술 2건의 원문 확보 핵심 추출	O	원문에서 기술 개요, 범위, 한계 추출
📊 시장 평가 에이전트	시장성, 채택 현황 조사	O	시장 규모, 상용화/채택 사례, 성장 전망 검색
🤝 이해관계자 평가 에이전트	관계자별 반응 조사	X	경쟁사 반응, 개발자 평가, 투자/업계 시각 등 검색
🏭 도메인 평가 에이전트	적용 도메인별 적합성	O	데이터센터, 온디바이스, 클라우드 서빙 등에서의 평가
⚖️ 평가 종합 에이전트	관점별 의견 종합 및 비교	X	관점 간 일치/불일치 의견 기반으로 의견 종합
📝 보고서 생성 에이전트	평가 보고서 생성	X	단계별 내용을 연결하여 보고서 생성

B. 설계

- 기술 선정
  - SW, HW 각 진영에서 하나씩, 총 2개의 기술을 선정
  - 선정 방식은 아래 2가지 중 선택
    - (1안) 에이전트 기반 : 선정 자체를 에이전트가 수행하도록 설계
    - (2안) Human 기반 : 조가 직접 선정
- RAG 적용
  - 위 표에서 “RAG 여부” 컬럼에 “O”로 구분된 것 중, 최소 1개 에이전트에서는 RAG 기반으로 설계 및 개발
  - 필요 문서는 제시되는 풀에서 선택하되, 적용 문서 개수에 상관없이 총 200 페이지 한정으로 진행
- Embedding 모델
  - 경제성을 고려하여 오픈소스 임베딩을 반드시 적용
  - 후보군을 선정하고, 어떤 기준으로 최종 선택했는지 사유 명시

C. 평가 관점 및 기준

> 🙏 기술 평가는 “우열”을 판정하는 것이 아닙니다.
>
> - 하나의 기술이 관점에 따라 어떻게 다르게 인식되는가를 조사, 비교, 대조하는 것
> - 시장은 A를 이렇게 보고, 도메인 전문가는 이런 한계를 지적한다 라는 시각 유지

1. 기술 성숙도 관점 - TRL(Technology Readiness Level) 기반
   - 원래 NASA가 우주 기술의 개발 단계를 표준화하기 위해 만든 9단계 척도
   - 이후 반도체를 포함한 첨단 산업 전반에서 “이 기술이 지금 어느 단계까지 와 있는가”를 객관적으로 판단하는데 활용
   - SWOT는 강하다/약하다처럼 상대적이고 주관적인 반면, TRL은 ‘지금 몇 단계인가’라는 절대적 위치 제공
   - ▶ TRL 9단계 구조
   - ▶ TRL 공개 정보 기반 추정
   - 단, KV cache 기술은 논문 발표 시점과 실제 채택 간 시차 있음 → 공개 정보 기반 추정임을 반드시 명시
2. 시장성 관점
   - 시장 규모, 성장성 : 시장 리포트, 산업 뉴스 등
   - 상용화, 채택 현황 : 실제 도입 사례, 제품 출시 발표 등
   - 생태계 지지 : 지원 프레임워크, 표준화 동향 등
3. 이해관계자 관점
   - 경쟁 기술 진영 : 경쟁사 반응, 대응 기술
   - 도입 기업, 개발자 : 도입 의견, 채택 시 장벽 등
   - 투자 업계 : 투자 동향, 애널리스트/미디어 평가 등
4. 도메인 적용 관점
   - 동일 기술도 적용 환경에 따라 평가가 달라짐. 대표 도메인 예시 :
     - 데이터센터, 클라우드 : 대규모, 비용 민감
     - OnDevice AI : 자원 제약, 전력 민감
     - 장문맥 처리 어플리케이션 : 문맥 길이 민감
   - 특정 도메인을 선택하여 평가 진행 할 수 있음

5. 종합 의견
    - 1~4번 관점들의 의견을 종합하되, 관점 간 상충 지점이 명시적으로 드러나도록 함
    - 객관적이고 중립적인 관점에서 처리되도록 설계 반영

D. 그래프 설계(안)

- 순차 흐름 : 정보 수집 → 분석 → 평가 → 보고서
- ▶ Graph(안) :
- 병렬 실행(Fan-out) 설계 시 고려사항 :
    - 여러 에이전트가 State를 (거의) 동시에 갱신하게 됨
    - 관점별 결과를 분리된 키로 관리하는 방식으로 설계

E. 평가 보고서

- 평가 보고서는 KV cache 최적화 기술을 여러 관점에서 비교 평가한 결과를 전달하는 문서임
- 특정 기술을 추천하거나 우열을 판정하지 않고, 관점에 따라 평가가 어떻게 구분되는지를 비교하는 목적임
- 보고서 목차
    - 자유롭게 정의하되, 보고서 맨 앞에는 “SUMMARY”, 맨 마지막에는 “REFERENCE” 챕터를 구성함
    - 참고 목차 :
        - SUMMARY : 전체 평가 보고서의 핵심 요약 (개요 장표 아님). 1/2 페이지를 넘지 않도록 구성
        1. 분석 배경 : 왜 KV cache가 필요하고, 분석하는가
        2. 기술 선정 : 선정한 기술 2건과 선정 이유
        3. 기술 개요 : 각 기술의 핵심 접근 방향, 한계점 등
        4. 관점별 평가 : 시장, 이해관계자, 도메인 관점별 평가 결과
        5. 시사점 : 관점 따라 평가가 엇갈리는 부분 중심으로 정리
        6. 한계점 : 공개 정보 기반 추정의 한계, 확증편향 방지 위해 취한 조치
        - REFERENCE : 보고서를 작성하는데 실제로 활용한 자료 목록만 기재.
- REFERENCE 표기 형식
    - 특허 : 출원인(YYYY-MM). *특허명*, 특허번호/공개번호, URL
    - 논문 : 저자(YYYY). 논문제목. *학술지/학회명*, 권(호), 페이지.
    - 기타 (웹페이지) : 기관명 또는 작성자(YYYY-MM-DD). *제목*. 사이트명, URL
    - ▶ Reference Example

🗃️ Deliverables

- 설계 산출물
    - 아래 내용이 포함되도록 작성 (자유양식)
        - 대상 기술 : 비교하고자 하는 대상 기술 선정 (선정 사유 포함)
        - B. 설계 : 기술선정 방식, RAG 적용 대상, 선정한 Embedding 모델
        - C. 평가 관점 및 기준 : 선정한 특정 도메인 선정, 4가지 관점별 평가 대상 및 기준 정의
        - D. 그래프 설계(안) : State 설계(table), Graph 흐름 설계(mermaid)
        - E. 평가 보고서 : 보고서 목차(초안)
    - 설계 산출물 : `RAG-Design_{캠퍼스}-{X반}_{이름1+이름2+이름3+이름4+이름5+이름6}.pdf`
        - 캠퍼스 = {광주, 울산, 판교}
    - Path : 반별 채널, slack thread
    - Due : DAY 3, 10시까지
- 개발 산출물
    - Github : Link
        - README.md
            - 아래 샘플 파일 참고하여 명확하고 간결하게 작성
            - Contributors 섹션에는 개인별 수행 역할 작성. 단 PM, PL 역할은 포함하지 않음
            - ▶ Sample :
    - 평가 보고서 : `RAG-Output_{캠퍼스}_{X반}_{이름1+이름2+이름3+이름4+이름5+이름6}.pdf`
        - 캠퍼스 = {광주, 울산, 판교}
    - Path : 반별 채널, slack thread (= Git Link + PDF)
    - Due : DAY 3, 16시까지

- 발표
  - 개발 산출물 제출 마감 후, 조별로 작성하신 `README.md` 로 발표 진행 (별도 자료 고려하지 않음)
  - 조별로 어떤 부분을 차별점으로 두고 설계 및 개발하였는지로 10분간 발표
  - 발표 말미에는 아래 내용에 대한 커멘트 추가
    - 평가 보고서의 핵심 포인트
    - Lessons Learned

▼ (참고) 평가항목

대상	항목	내용	배점	총점
설계 산출물	문제 정의	• 조별로 선정한 도메인에 따라 문제 정의가 명확하고 구체적임	5	100
	Agent 설계	• 에이전트 역할 분리가 합리적 • 불필요한 에이전트 없이 명확한 책임을 가짐	15	
	RAG 설계	• RAG 적용 에이전트 선정이 적절한지 • 문서 선정 전략 및 활용 방식이 타당한지	15	
	Embedding 모델 선택	• 오픈소스 임베딩 모델 선택 이유 및 적용 전략이 합리적인지 • (e.g., 리더보드 상위 랭크 ← 적절하지 않음)	10	
	평가 기준 설계	• 4가지 관점별 평가 대상과 기준이 합리적으로 구성되어 있음	15	
	State Schema 설계	• State 구조가 Graph 흐름에 맞게 정의되어 있음 • 에이전트 간 데이터 흐름이 명확하게 고려되어 있음	15	
	Graph 설계	• Workflow, Loop, Branch 구조가 논리적으로 설계되어 있음 • 에이전트 간 협업 구조가 잘 표현되어 있음	15	
	보고서 구조 설계	• 보고서 목차 및 전달 구조가 목적에 맞게 논리적으로 구성됨	10	
개발 산출물	설계 구현 충실도	• 제출된 설계 문서 기준으로 • 에이전트 구조, Graph 흐름, State 구조가 코드에 반영되어 있음	15	100
	Agent 구현	• 역할별 에이전트가 분리되어 구현됨 • 흐름 제어가 논리적으로 부합되도록 구현됨	15	
	RAG Pipeline 구현	• 문서 로딩, 임베딩, 검색, 컨텍스트 활용 흐름이 정상적으로 구현됨	20	
	코드 구조 및 프로젝트 구성	• 디렉토리 구조, 모듈 분리, 실행 스크립트 등이 명확하게 구성	10	
	실행 결과 재현성	• 코드 실행 시 실제로 보고서가 생성되는가	10	
	Output - 보고서	• 생성된 보고서가 설계된 내용에 따라 생성되었는가 • 보고 목적에 부합하는가	20	
	Output - README	• README 파일에 프로젝트 목적, 구조, 실행방법 등을 명확하고 간결하게 설명하고 있는가	10	

🙏 Final Reminders

- AI 도구를 활용하여 코딩은 할 수 있으나, 가이드 항목과 align 될 수 있도록 꼼꼼하게 확인
- 코드 재현되지 않는 부분은 없는지 파일 점검

© 2025-2026. Keejoo Bae for SK AX / SKALA. All rights reserved.

This content is protected by copyright law. Unauthorized copying, distribution, or modifications are strictly prohibited.